

Exercise 3: Syntax Sugar

30 Points Possible

3/15/2026

Attempt 1



In Progress

NEXT UP: Submit Assignment



Add Comment

Unlimited Attempts Allowed

2/25/2026 to 3/18/2026

Details

In this assignment, you will implement a basic search/filter system to experiment with some interesting features of Python.

Setup/Submission

- Setup: [Search.py \(https://ufl.instructure.com/courses/555493/files/104912081?wrap=1\)](https://ufl.instructure.com/courses/555493/files/104912081?wrap=1)  [\(https://ufl.instructure.com/courses/555493/files/104912081/download?download_frd=1\)](https://ufl.instructure.com/courses/555493/files/104912081/download?download_frd=1) contains the provided code. You can use an online editor such as [OneCompiler](https://onecompiler.com/python)  [\(https://onecompiler.com/python\)](https://onecompiler.com/python) (save regularly!), or setup a local environment that can run Python `>= 3.12` (recommended).
- Submission: You will submit your updated `Search.py` file. You should only implement the given functions the surrounding code structure and logic should be left unchanged (but you may add additional tests as desired).

Python Overview

Python is a programming language named after Monty Python; not the snake. Beyond basic syntax, there are two notable features that need to be understood for this assignment:

- [List] Comprehensions (Part 1): Allows transforming iterable values using map/filter logic, e.g. `[num * num for num in nums if num % 2 == 0]`.
 - Note that just `(num for num in nums)` returns a *generator*, not a list, which is also a neat feature but beyond the scope of this assignment.
- Decorators (Part 2): Allows wrapping a function to customize it's behavior, e.g. `@cache def fib(n): ...`.
 - Note that when decorators have arguments, the precedence is `@(returnsDecorator("argument")) def function(): ...`, i.e. the "decorator" is really a function call that returns the real decorator used by `@`.

Part 1: Comprehensions

Implement the `query` functions on the provided `games` dataset using comprehensions. You may assume that all arguments passed are valid; no need for any additional validation (see Part 3). Ensure you do not use loops or builtin methods like `map/filter` - only comprehensions! You may use intermediate variables as well to help make this easier to manage, but the "ideal" structure is a method with a single return `[... for ... in ... if ...]` statement.

Function	Description	Examples
<code>queryAll</code>	Returns all games in the format returned by <code>format</code> .	<code>queryAll()</code> : 7 results
<code>queryByName</code>	Returns all games with names that <i>contain</i> a match for the given <code>regex</code> . See docs for the re module  (https://docs.python.org/3/library/re.html) , which has been imported for you.	<code>queryByName("Stardew Valley")</code> : 1 result <code>queryByName("\\d+")</code> : 2 results
<code>queryByPrice</code>	Returns all games with prices within the given <code>min</code> / <code>max</code> range, inclusive.	<code>queryByPrice(min=35.00)</code> : 3 results <code>queryByPrice(max=15.00)</code> : 3 results
<code>queryByRating</code>	Returns all games with ratings at least the given <code>min</code> value, inclusive. The rating of a game is the average of all of it's reviews ratings (<code>0</code> if no	<code>queryByRating(1)</code> : 3 results

	reviews).	
	Important: Do NOT implement rating within a separate function - this should be implemented as part of your list comprehension (as ugly as it may be...).	
<code>queryBy</code>	Combines the behavior of <code>queryByName</code> / <code>queryByPrice</code> / <code>queryByRating</code> into a single method.	<code>queryBy(nameRegex="C", priceMax=25.00)</code> : 2 results

Part 2: Decorators

Implement the `contract` decorator to allow validating preconditions of a function before invoking it. This is more of a "neat idea" than "good practice", but demonstrates what's possible with some creative liberty.

Function	Description	Examples
<code>contract</code>	Takes a single <code>condition</code> argument, and returns a decorator which has the following behavior: - Evaluates <code>condition</code> using <code>eval</code>, in a scope that includes: - All function parameters/arguments that would normally be accessible. Start with <code>inspect.signature()</code> (https://docs.python.org/3/library/inspect.html#inspect.signature) (provided) and follow through the docs to determine how to get a dict linking parameters to arguments. - Bonus: Other global/nonlocal variables that would normally be accessible as well. Global is pretty straightforward, nonlocal is quite tricky! - If the evaluated condition is falsey, return <code>False</code>. Otherwise, call and return the decorated function as normal.	<pre># Note: bool needs globals; 'not not' works too @contract("bool(argument)") def truthy(argument): return "Invoked" truthy("") # False (contract violation) truthy("string") # "Invoked"</pre>

Part 3: Validation

Using the implemented `contract` decorator, add validation for preconditions to the original query functions.

Function	Description	Examples
<code>queryByName</code>	The given <code>regex</code> cannot be empty.	<code>queryByName("")</code> : <code>False</code>
<code>queryByPrice</code>	The given <code>min</code> must be at least <code>0</code> and at most <code>max</code> .	<code>queryByPrice(min=-1)</code> : <code>False</code>
<code>queryByRating</code>	The given <code>min</code> must be at least <code>0</code> and at most <code>5</code> .	
<code>queryBy</code>	Same as above functions for the corresponding parameter, except <code>nameRegex</code> which is <code>None</code> when unused.	

Changelog

2/27	- Updated provided code (Search.py (https://ufl.instructure.com/courses/555493/files/104912081?wrap=1) ↓ (https://ufl.instructure.com/courses/555493/files/104912081/download?download_frd=1)) with the following fixes/changes: <code>Game</code> dataclass now defines <code>__str__</code> instead of having a separate <code>format(game)</code> function. - Fixed misspelling of "Portal 2" in <code>games</code> (fixes associated test failures). - Fixed test output order to match <code>games</code> (specifically, moved up <code>Roller Coaster Tycoon 3</code>). - Removed deprecated <code>review</code> helper function, which should not be used (nest comprehensions).
------	--

Choose a submission type